

NAVEGACIÓN

Librería `navigation`

Principales conceptos de la librería `navigation` para navegar entre diferentes pantallas:

- **Grafo de navegación**: describe todos los destinos posibles y sus conexiones entre ellos.
- **Destino**: nodo del grafo de navegación al que se puede navegar.
- **Ruta**: define un destino y los argumentos necesarios para navegar, sin especificar nada de la UI.

Librería `navigation`

1. Define las rutas.
2. Crea un `NavController` que proporciona toda la funcionalidad para navegar entre destinos.
3. Crea un composable `NavHost` que contiene el grafo de navegación.

```kotlin
// Route.kt en el mismo directorio de App.kt
@Serializable
sealed interface Route {

    @Serializable
    data object Carta : Route

    @Serializable
    data class DetallePlato(val platoId: Int) : Route
}
```

`@Serializable` Proceso de convertir datos usados por una aplicación en un formato que puede transferido a través de una red, o almacenados en una base de datos o en archivo

`data object` Usado para acciones SIN datos asociados.

`data class` Usado para acciones CON datos asociados.

```kotlin
@Serializable
sealed interface Route

@Serializable
data object Carta : Route

@Serializable
data class DetallePlato(val platoId: Int) : Route
```

`@Serializable`	Proceso de convertir datos usados por una aplicación en un formato que puede transferido a través de una red, o almacenados en una base de datos o en archivo
`data object`	Usado para acciones SIN datos asociados.
`data class`	Usado para acciones CON datos asociados.

- Crea un `NavController` que proporciona toda la funcionalidad para navegar entre destinos.
- Crea un composable `NavHost` que contiene el grafo de navegación.

```
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {
        composable<Route.Carta> { /* ... */ }
        composable<Route.DetallePlato> { /* ... */ }
    }
}
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

```
@Serializable
data class DetallePlato(val platoId: Int) : Route
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

```
@Serializable
data class DetallePlato(val platoId: Int) : Route
```

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

Navegar entre destinos

¿Por qué solo pasamos el ID en vez del plato entero?

Al navegar entre destinos, es recomendable pasar la información mínima innecesaria. De esta manera, prevenimos que se pierdan datos durante los cambios de la configuración o al actualizar los objetos referenciados.

Mejor pasamos la ID y luego volvemos a buscar la información en nuestra base de datos.

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

Recibir el dato en el destino

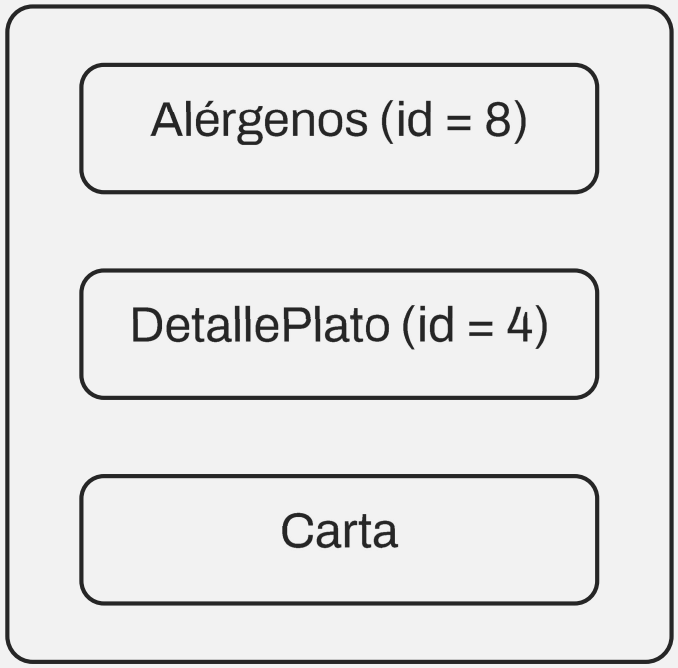
```
composable<Route.DetallePlato> { it ->
        val route: Route.DetallePlato = it.toRoute()
        // route.platoId
}
```

Recibir el dato en el destino

```
composable<Route.DetallePlato> { backStackEntry ->
        val route: Route.DetallePlato = backStackEntry.toRoute()
        // route.platoId
}
```

La pila (*stack*)

Cuando navegas en tu app, *Compose Navigation* mantiene una **pila** (*stack*) con el historial de pantallas por las que has pasado



`startDestination`: Define la primera pantalla que verá el usuario al iniciar la app

```
NavHost(navController = navController, startDestination = Route.Carta) {
    composable<Route.Carta> { /* ... */ }
    composable<Route.DetallePlato> { /* ... */ }
}
```

`navController.navigate(…)`: Añade un nuevo destino arriba de la pila

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

`popBackStack()`: Elimina el destino de arriba y vuelve al anterior

```
Button(onClick = { navController.popBackStack() }) {
    Text("Volver a la carta.")
}
```

¿Cómo rellenaríamos esto con los que sabemos hasta ahora?

```
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {
        composable<Route.Carta> { /* ... */ }
        composable<Route.DetallePlato> { /* ... */ }
    }
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
class CartaViewModel(
    private val obtenerPlatos: ObtenerPlatosUseCase
) : ViewModel() {

    private val _state = MutableStateFlow(CartaState())
    val state: StateFlow<CartaState> = _state.asStateFlow()

    init {
        viewModelScope.launch {
            _state.update { it.copy(platos = obtenerPlatos()) }
        }
    }
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
@Composable
fun PizzaScreenRoot(viewModel: CamareroViewModel) {
    val state by viewModel.state.collectAsStateWithL

    PizzaScreen(
        state = state,
        onAction = viewModel::onAction
    )
}
```

```
@Composable
fun CartaScreenRoot(
    viewModel: CartaViewModel,
    onPlatoClick: (Plato) -> Unit
) {
    val state by viewModel.state.collectAsStateWithLifecycle()

    CartaScreen(
        state = state,
        onAction = { action ->
            when (action) {
                is CartaAction.OnPlatoClick -> onPlatoClick(action.plato) // navegación
                else -> viewModel.onAction(action) // resto: al ViewModel
            }
        }
    )
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

```
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.popBackStack() }
    )
}
```

```
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.popBac
    )
}
```

```
class DetallePlatoViewModel(
    private val platoId: Int,
    private val obtenerPlatoPorId: ObtenerPlatoPorIdUseCase
) : ViewModel() {

    private val _state = MutableStateFlow(DetallePlatoState())
    val state: StateFlow<DetallePlatoState> = _state.asStateFlow()

    init {
        cargarPlato()
    }

    private fun cargarPlato() {
        viewModelScope.launch {
            _state.update { it.copy(cargando = true) }
            val plato = obtenerPlatoPorId(platoId)
            _state.update { it.copy(cargando = false, plato = plato) }
        }
    }
}
```

```
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.p
    )
}
```

```
@Composable
fun DetallePlatoScreenRoot(
    viewModel: DetallePlatoViewModel
    onVolver: () -> Unit
) {
    val state by viewModel.state.collectAsStateWithLifecycle()

    DetallePlatoScreen(
        state = state
        onAction = { action ->
            is DetallePlatoAction.OnVolver -> onVolver()
            else -> viewModel.onAction(action)
        }
    )
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```